

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 2

Disciplina: Tehnici de programare

Tema: ” Lucrul în linia de comandă Linux ”

A efectuat :

st. gr.SI – 266 Fratea Mihaela

A verificat :

asistent universitar Cristian Goncari

Chișinău – 2026

REGULI DE PREDARE A LUCRĂRII

Citește această pagină înainte de a începe. Nerespectarea regulilor de mai jos înseamnă că lucrarea nu poate fi identificată și nu este primită.

1. Cum se numește fișierul pe care îl trimiți

Denumirea fișierului trebuie să respecte exact acest tipar:

L2_TP_Nume_Prenume_Grupa.docx

Exemplu, pentru studentul Popescu Ion din grupa SI-266:

L2_TP_Popescu_Ion_SI-266.docx

Partea	Ce scrii	Exemplu
L2	Litera L și numărul laboratorului. La această lucrare este L2.	L2
TP	Abrevierea disciplinei, cu majuscule. Se scrie identic.	TP
Nume	Numele de familie, cu majusculă la început, fără diacritice.	Popescu
Prenume	Prenumele, cu majusculă la început, fără diacritice.	Ion
Grupa	Grupa, cu majuscule și cu cratimă.	SI-266
.docx	Extensia. Numai Word. Nu se acceptă PDF.	.docx

2. Cum trimiți lucrarea

	Regula
Adresa de email	cristian.gonceari@isa.utm.md
Subiectul emailului	L2_TP_Popescu_Ion_SI-266
	Adică exact denumirea fișierului, dar fără „.docx”. Nimic altceva în subiect.
Atașament	Un singur fișier .docx. Fără arhive, fără poze separate.
Corpul emailului	Nu este nevoie să scrii nimic.

De ce contează subiectul: cu acest subiect, toate lucrările unui laborator pot fi găsite și sortate instantaneu. Un subiect scris liber înseamnă că lucrarea ta se poate pierde printre celelalte.

3. Cum completezi acest document

1. Completează pagina de titlu. Tema lucrării este stabilită de profesor și nu se modifică. Tu înlocuiești doar textul evidențiat cu galben — grupa, numele și prenumele tău — apoi ștergi evidențierea galbenă.
2. Parcurge sarcina lucrării de la pagina următoare, pe platforma pwn.college.
3. Răspunde la toate cele 20 de întrebări, în chenarele pregătite sub fiecare întrebare. Scrie cu cuvintele tale.
4. Completează secțiunea de concluzii.
5. Salvează fișierul cu denumirea de la punctul 1 și trimite-l pe email.

SARCINA LUCRĂRII

Parcurge pe platforma pwn.college dojo-ul Linux Luminarium și rezolvă integral modulele obligatorii: Hello Hackers, Pondering Paths, Comprehending Commands, Digesting Documentation, File Globbing și Practicing Piping. Rezolvă apoi cel puțin patru dintre modulele recomandate: Shell Variables, Data Manipulation, Processes and Jobs, Untangling Users și Perceiving Permissions. Pe parcursul lucrului notează-ți comenzile exacte pe care le-ai rulat, mesajele de eroare primite și modul în care ai depășit fiecare blocaj, fiindcă întrebările din acest raport se referă la experiența ta directă, nu la definiții. Răspunde apoi, în acest document, la toate cele 20 de întrebări, cu propriile cuvinte și cu exemple concrete din sesiunile tale de lucru. Răspunsurile copiate din manuale sau de pe internet, care nu au legătură cu ceea ce ai rulat efectiv, nu se punctează.

Platforma	Cursul de parcurs	Adresa
pwn.college	Linux Luminarium	https://pwn.college/linux-luminarium/

Modulele pe care le parcurgi

Nr.	Modul	Challenge-uri	De rezolvat	Nivel
1	Hello Hackers	3	3	OBLIGATORIU
2	Pondering Paths	8	8	OBLIGATORIU
3	Comprehending Commands	15	15	OBLIGATORIU
4	Digesting Documentation	7	7	OBLIGATORIU
5	File Globbing	10	10	OBLIGATORIU
6	Practicing Piping	15	15	OBLIGATORIU
7	Shell Variables	8	8	RECOMANDAT
8	Data Manipulation	6	6	RECOMANDAT
9	Processes and Jobs	10	10	RECOMANDAT
10	Untangling Users	4	4	RECOMANDAT
11	Perceiving Permissions	8	8	RECOMANDAT
12	Chaining Commands	12	12	BONUS
13	Terminal Multiplexing	6	6	BONUS
14	Pondering PATH	5	5	BONUS
15	Silly Shenanigans	6	6	BONUS
16	Daring Destruction	5	5	BONUS

Modulele marcate OBLIGATORIU conțin materia pe care se bazează întrebările. Cele marcate RECOMANDAT și BONUS te ajută la întrebările mai dificile.

ÎNTREBĂRI

Răspunde la toate cele 20 întrebări, în chenarele de mai jos. Fiecare răspuns corect valorează 0,5 puncte.

1. La exercițiul Intro to Arguments ai rulat comanda hello mai întâi fără argument, apoi cu argumentul hackers. Explică ce face shell-ul concret cu textul pe care îl scrii la prompt înainte de a porni programul și de ce programul a reacționat diferit în cele două cazuri.

(modulul Hello Hackers)

Când am rulat hello fără argument și apoi hello hackers, shell-ul a împărțit textul scris de mine în cuvinte, după spații. Primul cuvânt l-a luat ca nume al programului, iar restul ca argumente pe care i le-a dat programului la pornire.

Programul a reacționat diferit pentru că în al doilea caz avea un argument de citit, iar în primul nu avea niciunul.

2. La exercițiul Command History flag-ul a fost introdus în istoricul shell-ului tău. Relatează concret cum ai ajuns la el: ce ai apăsăat sau ce comandă ai folosit, ce a apărut pe ecran și unde se păstrează acest istoric.

(modulul Hello Hackers)

Am ajuns la flag prin istoricul shell-ului prin săgeata în sus. Pe ecran a apărut istoricul comenzilor trecute. Istoricul e ținut de bash în memorie cât timp sesiunea e deschisă, iar la ieșire se salvează în fișierul “~/.bash_history”.

3. Aflându-te chiar în directorul /challenge, comanda scrisă doar cu numele programului eșuează, iar varianta cu ./ în față funcționează. Explică ce caută shell-ul în fiecare dintre cele două cazuri.

(modulul Pondering Paths)

În /challenge, fără ./ comanda a eșuat pentru că shell-ul caută un nume simplu doar în directoarele din PATH, iar directorul curent nu e printre ele. Cu ./ i-am spus direct că programul e în directorul curent, așa că nu a mai căutat în PATH.

4. La exercițiile implicit relative paths, from / și implicit relative path a trebuit să rulezi același program o dată din directorul / și o dată din /challenge, scriind de fiecare dată altceva. Explică de la ce punct pornește sistemul când primește o cale relativă și cum se schimbă calea pe care trebuie să o scrii atunci când te muți cu cd.

(modulul Pondering Paths)

Când primește o cale relativă, sistemul pornește din directorul în care mă aflu. Din / a trebuit să scriu [calea din /], iar din /challenge doar [calea scurtă]. Calea depinde de directorul curent, deci după fiecare cd trebuie s-o scriu altfel.

5. La exercițiul home sweet home aveai voie să dai un argument de cel mult trei caractere. Scrie exact ce ai introdus ca argument și explică prin ce mecanism acele câteva caractere ajung să indice un fișier din directorul tău personal.

(modulul *Pondering Paths*)

Ca argument am introdus ~/a. Shell-ul înlocuiește ~ cu calea directorului meu personal înainte să pornească programul, așa că programul a primit calea completă și a găsit fișierul din home.

6. Numele comenzii cat vine de la concatenare. Pe baza exercițiilor pe care le-ai rulat în Comprehending Commands, explică ce se întâmplă când îi dai mai multe fișiere ca argumente și cum se vede acest lucru în ieșirea pe care ai obținut-o.

(modulul *Comprehending Commands*)

Când i-am dat mai multe fișiere, cat le-a citit pe rând și le-a afișat unul după altul, fără nicio separare. În ieșire am văzut conținutul lor lipit, ca un singur text:

```
hacker@commands~cat-not-the-pet-but-the-command:~$ cat flag flag
pwn.college{oQilLIS1S6X49AObL6htzEqQPRk.QXxcTN0wCO1YTMylzW}
pwn.college{oQilLIS1S6X49AObL6htzEqQPRk.QXxcTN0wCO1YTMylzW}
```

7. La exercițiul An Epic Filesystem Quest ai urmărit indicii dintr-un director în altul până la flag. Relatează concret traseul tău: ce pereche de comenzi ai repetat la fiecare pas și cum ai decis încotro mergi mai departe.

(modulul *Comprehending Commands*)

La fiecare pas am repetat ls ca să văd ce e în director și cat ca să citesc indiciul, apoi am folosit cd în directorul pe care îl indica indiciul. Traseul meu a fost prin directoare precum /usr/lib/python3/dist-packages/cryptography/hazmat/bindings/__pycache__, /usr/lib/x86_64-linux-gnu/perl/5.38.2/IO/Socket, /usr/share/doc/python3-markupsafe și /usr/lib/x86_64-linux-gnu/perl-base/unicore/lib/XIDS.J. Decizia de a merge mai departe s-a bazat mereu pe ce spunea textul indiciului: fie calea directorului următor, fie ce comportament special avea (delayed/trapped/hidden).

8. La exercițiul linking files ai creat o legătură simbolică cu ln -s. Explică ce conține de fapt fișierul-legătură și ce se întâmplă în momentul în care rulezi cat pe el.

(modulul Comprehending Commands)

Fișierul creat cu ln -s conține doar calea către fișierul-țintă, nu conținutul lui. Când am rulat cat pe el, sistemul a urmat calea și mi-a afișat conținutul țintei, adică a afișat conținutul fișierului flag.

9. Compară cele trei surse de documentație folosite în Digesting Documentation: man, opțiunea --help și comanda help. Spune pentru ce fel de comandă se potrivește fiecare și de ce pentru comanda-builtin din ultimul exercițiu nu ai putut folosi man.

(modulul Digesting Documentation)

man e pentru programele externe și dă manualul complet. --help afișează un rezumat scurt, direct de la program. help e pentru comenzile builtin ale shell-ului. La comanda din ultimul exercițiu nu am putut folosi man pentru că e builtin, adică face parte din bash și nu e un program separat cu pagină de manual proprie.

10. La exercițiul Searching For Manuals pagina de manual avea un nume generat aleator. Spune ce comandă ai folosit ca să o găsești, după ce cuvânt ai căutat și cum arăta rezultatul afișat de acea căutare.

(modulul Digesting Documentation)

Am căutat pagina de manual cu man -k flag. Rezultatul a fost o listă de linii de forma nume – ivmkanw - descriere, printre care am primit și flag-ul.

11. Explică cine face expandarea unui tipar precum /challenge/f*: shell-ul sau programul apelat. Argumentează pe baza a ceea ce ai observat efectiv, inclusiv ce s-a întâmplat când niciun fișier nu se potrivea tiparului.

(modulul File Globbing)

Expandarea o face shell-ul, înainte de pornirea programului, care primește deja lista de fișiere. Am văzut asta când am încercat /challenge/f*. Când niciun fișier nu s-a potrivit, tiparul a rămas scris exact așa, iar programul a primit un nume care nu există și a dat eroarea “No such file or directory”.

12. Compară folosirea unui glob cu folosirea completării prin tasta Tab pentru a ajunge la același fișier. Spune în ce situație concretă din modul ai preferat una și în ce situație cealaltă, și de ce completarea prin Tab este considerată mai sigură.

(modulul File Globbing)

Globul se potrivește cu toate fișierele care respectă tiparul, deci poate prinde și fișiere pe care nu le voiam. Cu Tab văd numele complet exact înainte să apăs Enter, de aceea e mai sigur. La exercițiul “Matching with ?” am preferat globul pentru că doar trebuia să înlocuiesc “c” și “l” cu “?”, iar la Tab completion am preferat Tab pentru că trebuia doar să apăs tasta TAB pentru a completa provocarea.

13. Descrie ce se întâmplă concret cu textul dintre două programe legate prin pipe, pe exemplul pe care l-ai rulat tu la exercițiul Grepping live output. Precizează de unde citește al doilea program și dacă textul mai trece prin vreun fișier de pe disc.

(modulul Practicing Piping)

La Grepping live output, ieșirea primului program a mers direct în al doilea, iar grep a citit de la intrarea lui standard, nu dintr-un fișier. Textul a trecut printr-un pipe, care e o zonă din memorie, deci nu a fost scris pe disc.

14. Ai redirectionat ieșirea programului /challenge/run într-un fișier folosind operatorul > și totuși pe ecran au continuat să apară mesaje. Explică de ce au apărut acele mesaje și ce ai scris ca să le capturezi și pe ele.

(modulul Practicing Piping)

Mesajele au rămas pe ecran pentru că > redirectionează doar ieșirea standard, iar mesajele erau pe ieșirea de erori. Ca să le prind și pe ele am scris /challenge/run > myflag.

15. Modulul îți cere mai întâi să salvezi ieșirea într-un fișier și apoi să o filtrezi cu grep, iar la exercițiul următor să filtrezi direct prin pipe. Compară cele două metode și spune în ce situație reală ai alege salvarea în fișier și în ce situație pipe-ul direct.

(modulul Practicing Piping)

Prima metodă (în fișier, apoi grep) e bună când vreau să păstrez ieșirea și să o filtrez de mai multe ori fără să rulez programul din nou. Pipe-ul direct îl aleg când am nevoie de un rezultat rapid, o singură dată, fără fișiere în plus.

16. La exercițiul Exporting Variables ai setat o variabilă, iar programul lansat de tine nu o vedea până când nu ai folosit export. Explică diferența dintre o variabilă de shell obișnuită și una exportată, din perspectiva procesului-copil.

(modulul Shell Variables)

Am ajuns la flag folosind `export PWN=COLLEGE` (variabilă exportată) și `COLLEGE=PWN` (fără export). Rulând `/challenge/run`, acesta a confirmat că a primit PWN dar nu și COLLEGE, apoi a dat flag-ul. Asta arată că doar variabila exportată ajunge în mediul procesului-copil, cea neexportată rămâne doar în shell-ul meu.

17. La exercițiile din Data Manipulation flag-ul îți venea stricat, cu caractere în plus, cu linii noi intercalate sau cu majusculele inversate. Alege unul dintre aceste exerciții, scrie comanda exactă pe care ai folosit-o pentru a repara ieșirea și spune cum arăta textul înainte și după.

(modulul Data Manipulation)

Am ales exercițiul [Deleting characters]. Comanda a fost `/challenge/run | tr -d '^%'`, iar înainte de filtrare ieșirea conținea flag-ul amestecat cu ^ și % intercalate; după `tr -d`, acestea au dispărut și a rămas doar textul flag-ului.

18. Ai apăsă Ctrl-Z peste un program care rula. Explică în ce stare ajunge procesul imediat după această apăsare și de ce el nu își continuă lucrul până când nu dai comanda bg sau fg.

(modulul Processes and Jobs)

După Ctrl-Z procesul e oprit, adică rămâne în memorie, dar nu mai primește timp de procesor. Nu își continuă lucrul până când nu primește semnalul de reluare, pe care îl trimite cu `bg` (rulează în fundal) sau `fg` (revine în prim-plan).

19. La exercițiul The SUID Bit ai adăugat bitul SUID unui program și abia apoi el a putut citi flag-ul. Explică cu ce identitate rulează un program obișnuit față de unul cu bit SUID și de ce asta schimbă rezultatul verificării drepturilor la citirea fișierului.

(modulul Perceiving Permissions)

Un program obișnuit rulează cu identitatea mea, deci verificarea drepturilor se face după utilizatorul meu. Cu bit SUID, programul rulează cu identitatea proprietarului fișierului, așa că sistemul a verificat drepturile acestuia, care avea voie să citească flag-ul.

20. În modulul Untangling Users ai schimbat identitatea o dată cu su și o dată cu sudo. Explică ce parolă ți s-a cerut în fiecare dintre cele două cazuri și pe ce se bazează sistemul când decide dacă îți permite sau nu operația.

(modulul Untangling Users)

La su mi s-a cerut hack-the-planet, iar la sudo nu mi-a cerut o parolă. La sudo, sistemul decide după configurarea din /etc/sudoers, adică dacă am voie să rulez comanda respectivă.

CONCLUZII

Scrie 10 – 15 rânduri: ce ai învățat concret, ce ți s-a părut cel mai dificil, unde te-ai blocat și cum ai ieșit din blocaj. Evită formulările generale de tipul „lucrarea a fost utilă”.

Din această lucrare am înțeles concret cum shell-ul interpretează o comandă înainte s-o execute: expandează ~ și globurile (*, ?) în argumente reale, apoi caută programul fie în PATH (dacă scriu doar numele), fie direct la calea dată (cu ./ sau o cale absolută). Am văzut și diferența dintre stdout și stderr la exercițiul cu /challenge/run > myflag, unde mesajele au rămas pe ecran pentru că veneau pe descriptorul 2, nu pe 1, deci > nu le-a prins.

Cel mai dificil mi s-a părut exercițiul cu opțiunea căutată în man challenge: am identificat un cuvânt din manual (ivmkanw), dar comanda nu funcționa, pentru că nu citisem corect linia întreagă din pagina de manual, opțiunea reală având un prefix (-/--) pe care nu-l inclusesem. Am ieșit din blocaj redeschizând manualul, căutând din nou cu /, și citind atent formatul exact al opțiunii, apoi verificând cu --help lista de opțiuni valide înainte s-o rulez.

M-am mai blocat puțin la diferența dintre variabile exportate și neexportate (export PWN=COLLEGE față de COLLEGE=PWN), pentru că la prima încercare nu eram sigur dacă ordinea comenzilor contează; am rezolvat setând clar variabilele separat și verificând cu env | grep ce ajunge efectiv la procesul-copil.

Per total, exercițiile cu linkuri simbolice, SUID (su vs sudo) și redirectarea stdout/stderr mi-au clarificat cel mai bine cum Linux separă identitatea utilizatorului, permisiunile fișierelor și fluxurile de date ale unui proces.

BIBLIOGRAFIE

1. pwn.college (Arizona State University), Linux Luminarium, [Online]. Disponibil: <https://pwn.college/linux-luminarium/>
2. pwn.college, linux-luminarium (codul sursă și descrierile modulelor și ale challenge-urilor), GitHub, [Online]. Disponibil: <https://github.com/pwncollege/linux-luminarium>
3. Free Software Foundation, GNU Bash Reference Manual, [Online]. Disponibil: <https://www.gnu.org/software/bash/manual/bash.html>
4. M. Kerrisk, The Linux man-pages project (paginile de manual ale comenzilor Linux), [Online]. Disponibil: <https://man7.org/linux/man-pages/>
5. W. Shotts, The Linux Command Line: A Complete Introduction, No Starch Press, [Online]. Disponibil: <https://linuxcommand.org/tlcl.php>
6. The Linux Foundation, Filesystem Hierarchy Standard (FHS), versiunea 3.0, [Online]. Disponibil: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html

BAREM DE NOTARE

Nota se calculează numai din răspunsurile la cele 20 de întrebări.

$20 \text{ întrebări} \times 0,5 \text{ puncte} = 10 \text{ puncte} = \text{nota } 10$

Cum se punctează un răspuns

Punctaj	Când se acordă
0,5 puncte	Răspuns corect și complet: explică mecanismul, nu doar enunță rezultatul.
0,25 puncte	Răspuns parțial corect: ideea principală este bună, dar explicația este incompletă sau conține o imprecizie.
0 puncte	Răspuns greșit, lipsă, sau copiat cuvânt cu cuvânt de la alt student ori dintr-o sursă de pe internet.

Cum se transformă punctajul în notă

Răspunsuri corecte	Punctaj	Nota
20 răspunsuri corecte	10 puncte	10
18 – 19 răspunsuri corecte	9 – 9,5 puncte	9
16 – 17 răspunsuri corecte	8 – 8,5 puncte	8
14 – 15 răspunsuri corecte	7 – 7,5 puncte	7
12 – 13 răspunsuri corecte	6 – 6,5 puncte	6
10 – 11 răspunsuri corecte	5 – 5,5 puncte	5
sub 10 răspunsuri corecte	sub 5 puncte	nepromovat — lucrarea se reface